

Lecture 4: Data Lab, importing and transforming data with dplyr (rows)

ØKA2015 Applied Data Science, University of Inland Norway

Touseef Hameed

18 September 2026

Contents

1	What this document is	1
2	Packages and the data	2
3	Quick inspection: look before you touch	2
4	Operators, the pipe, and <code>na.rm</code>	3
5	Rows: <code>filter()</code>	6
6	Rows: <code>arrange()</code>	8
7	Rows: <code>distinct()</code> and <code>count()</code>	10
8	Check yourself	11
9	What you can do now	11
10	Sources	11

1 What this document is

This is the Lecture 4 Data Lab as a written tutorial, in the same order as the script `lecture04_wrangling.R` and as Chapter 3 of *R for Data Science* up to the end of “Rows”. The explanation is text between the code instead of comments beside it. Read it after the lab, or run it: open the `.Rmd` file in RStudio, press **Knit**, and this PDF is rebuilt on your computer.

Lecture 3 brought a dataset into R and looked at it. Today you start changing it. You rarely get data in exactly the form you need; you keep some rows, reorder them, and count what is there. Those are the *row verbs* of the **dplyr** package. Columns and groups follow in the next lectures.

2 Packages and the data

Two packages: the **tidyverse**, which provides `dplyr` (the verbs, and `glimpse()`) and `readr` (`read_csv()`), and **nycflights13**, which contains the data. Install each once per computer. Running the install again does no harm; it only takes time.

```
install.packages("tidyverse")
install.packages("nycflights13")
```

Then load both, every session:

```
library(tidyverse)
library(nycflights13)
flights <- nycflights13::flights
```

`library(tidyverse)` prints a *Conflicts* note: `dplyr::filter()` masks `stats::filter()`. Two packages have a function called `filter`, and the tidyverse one is used. The double colon, `packageName::function()`, says precisely which one you mean. `nycflights13::flights` reads as “flights from the package nycflights13”.

On Wednesday the data came from a CSV file. A package that ships its data is a fourth way to get data into R. Everything that follows is the same whichever way you used.

3 Quick inspection: look before you touch

`flights` is a tibble: 336,776 flights that left New York in 2013, 19 columns. A tibble prints only the first rows and the columns that fit. Four ways to see more: `View(flights)` opens the spreadsheet viewer in RStudio, `print(flights, width = Inf)` prints every column, `print(flights, n = Inf)` prints every row (all 336,776, so use it with care), and `glimpse()` lists every column with its type.

```
flights
```

```
#> # A tibble: 336,776 x 19
#>   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
#>   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
#> 1  2013     1     1     517             515         2     830             819
#> 2  2013     1     1     533             529         4     850             830
#> 3  2013     1     1     542             540         2     923             850
#> # i 336,773 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#> #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#> #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
nrow(flights)
```

```
#> [1] 336776
```

```
ncol(flights)
```

```
#> [1] 19
```

```
glimpse(flights)
```

```
#> Rows: 336,776
#> Columns: 19
#> $ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
#> $ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ~
#> $ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ~
#> $ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -1~
#> $ arr_time  <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849,~
#> $ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851,~
#> $ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -1~
#> $ carrier   <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "~
#> $ flight    <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4~
#> $ tailnum   <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394~
#> $ origin    <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA",~
#> $ dest      <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD",~
#> $ air_time  <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1~
#> $ distance  <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ~
#> $ hour      <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6~
#> $ minute    <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0~
#> $ time_hour <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-01 0~
```

The type under each name decides what you can do with it: `<int>` and `<dbl>` are numbers, `<chr>` is text, `<dtm>` a date-time. Delays are in minutes, negative means early, and `dep_time` 517 means 05:17. NA in `dep_delay` means the flight was cancelled.

`$` pulls one column out as a plain vector, which goes straight into `mean()`, `sum()` or `max()`:

```
head(flights$dep_delay)
```

```
#> [1] 2 4 2 -1 -6 -4
```

```
mean(flights$distance)
```

```
#> [1] 1039.913
```

4 Operators, the pipe, and `na.rm`

A condition compares values and gives TRUE or FALSE. Two equals signs ask “is it equal?”. Text is compared exactly, capitals included.

```
3 == 3
```

```
#> [1] TRUE
```

```
3 != 4
```

```
#> [1] TRUE
```

```
5 >= 2
```

```
#> [1] TRUE
```

```
"JFK" == "jfk"
```

```
#> [1] FALSE
```

Logical operators combine TRUE and FALSE: & is AND, | is OR, ! is NOT. %in% asks whether a value is one of a set, and a:b is a range.

```
TRUE & FALSE
```

```
#> [1] FALSE
```

```
TRUE | FALSE
```

```
#> [1] TRUE
```

```
!TRUE
```

```
#> [1] FALSE
```

```
"JFK" %in% c("JFK", "LGA", "EWR")
```

```
#> [1] TRUE
```

```
7:9
```

```
#> [1] 7 8 9
```

The pipe. `x |> f(y)` means `f(x, y)`: take `x`, *then* apply `f`. Read `|>` as “then”. **Ctrl + Shift + M** inserts it in RStudio; if that gives `%>%`, the older pipe, tick *Use native pipe operator* under Tools, Global Options, Code. The two behave the same for everything in this course.

```
sqrt(16)
```

```
#> [1] 4
```

```
16 |> sqrt()
```

```
#> [1] 4
```

```
flights |> nrow()
```

```
#> [1] 336776
```

The pipe is useful when there are several steps, because each line reads as a step in a sentence: take the flights, then keep January, then average the departure delay.

```
flights |>
  filter(month == 1) |>
  summarise(avg_delay = mean(dep_delay))
```

```
#> # A tibble: 1 x 1
#>   avg_delay
#>       <dbl>
#> 1      NA
```

The answer is NA. `mean()`, `sum()` and `max()` return NA if any value is missing, and cancelled flights have no departure delay. `na.rm = TRUE` tells the function to skip the missing values for this one calculation; the data itself is unchanged.

```
v <- c(1, 2, NA, 4)
mean(v)
```

```
#> [1] NA
```

```
mean(v, na.rm = TRUE)
```

```
#> [1] 2.333333
```

```
flights |>
  filter(month == 1) |>
  summarise(avg_delay = mean(dep_delay, na.rm = TRUE))
```

```
#> # A tibble: 1 x 1
#>   avg_delay
#>       <dbl>
#> 1    10.04
```

Note the two different signs in that code. Inside `summarise()`, `=` gives the result a name, `avg_delay`. Outside a function call, `<-` stores an object, as in `v <- ....`. They are not interchangeable: `=` names something inside a call, `<-` creates an object.

What every dplyr verb shares. The first argument is a data frame; the pipe supplies it. The next arguments name columns, without quotes. The result is a new data frame, and the input is never changed. That is why these two lines are the same command. Here `|> nrow()` is added so that only the row count prints; without it you get the whole table.

```
filter(flights, month == 1) |> nrow()
```

```
#> [1] 27004
```

```
flights |> filter(month == 1) |> nrow()
```

```
#> [1] 27004
```

5 Rows: filter()

`filter()` keeps the rows for which a condition is TRUE. All the columns stay.

```
flights |> filter(dep_delay > 120)
```

```
#> # A tibble: 9,723 x 19
#>   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
#>   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
#> 1  2013     1     1     848             1835         853     1001             1950
#> 2  2013     1     1     957             733          144     1056             853
#> 3  2013     1     1    1114             900          134     1447             1222
#> # i 9,720 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#> #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#> #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

9,723 of 336,776 flights left more than two hours late. Conditions combine with `&` (and), `|` (or) and `%in%` (is one of). Only the row counts are printed below; each line returns a whole table like the one above.

```
flights |> filter(month == 1 & day == 1) |> nrow()
```

```
#> [1] 842
```

```
flights |> filter(month == 1 | month == 2) |> nrow()
```

```
#> [1] 51955
```

```
flights |> filter(month %in% c(1, 2)) |> nrow()
```

```
#> [1] 51955
```

```
flights |> filter(dest == "IAH") |> nrow()
```

```
#> [1] 7198
```

January and the first: New Year's Day, 842 flights. January or February, written two ways: 51,955 flights. Houston, "IAH" in quotes because it is text: 7,198 flights.

`is.na()` is a condition like any other: TRUE where a value is missing. With ! in front it keeps the rows that have a value.

```
flights |> filter(is.na(dep_delay)) |> nrow()
```

```
#> [1] 8255
```

```
flights |> filter(!is.na(dep_delay)) |> nrow()
```

```
#> [1] 328521
```

8,255 cancelled flights, and 328,521 that departed.

Two common mistakes. The first stops with a helpful message:

```
flights |> filter(month = 1)
```

```
#> Error in 'filter()':  
#> ! We detected a named input.  
#> i This usually means that you've used '=' instead of '=='.  
#> i Did you mean 'month == 1'?
```

The second gives no error and is wrong: R reads `month == 1 | 2` as `(month == 1) | 2`, and the number 2 counts as TRUE, so every row passes.

```
flights |> filter(month == 1 | 2) |> nrow()
```

```
#> [1] 336776
```

Write `month == 1 | month == 2`, or `month %in% c(1, 2)`.

A verb returns a new table and changes nothing. To keep a result, name it with `<-`, and a second object appears in the Environment pane:

```
jan1 <- flights |> filter(month == 1 & day == 1)
nrow(flights)
```

```
#> [1] 336776
```

```
nrow(jan1)
```

```
#> [1] 842
```

6 Rows: arrange()

`arrange()` changes the order of the rows. Ascending by default, descending with `desc()`. The row count never changes.

```
flights |> arrange(dep_delay)
```

```
#> # A tibble: 336,776 x 19
#>   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
#>   <int> <int> <int>   <int>         <int>      <dbl>   <int>         <int>
#> 1  2013    12     7    2040           2123      -43     40           2352
#> 2  2013     2     3    2022           2055     -33    2240           2338
#> 3  2013    11    10    1408           1440     -32    1549           1559
#> # i 336,773 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#> #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#> #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
flights |> arrange(desc(dep_delay))
```

```
#> # A tibble: 336,776 x 19
#>   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
#>   <int> <int> <int>   <int>         <int>      <dbl>   <int>         <int>
#> 1  2013     1     9     641           900    1301    1242           1530
#> 2  2013     6    15    1432          1935    1137    1607           2120
#> 3  2013     1    10    1121          1635    1126    1239           1810
#> # i 336,773 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#> #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#> #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

The longest departure delay was 1,301 minutes, on 9 January.

With several columns, the first column decides the order and each later column breaks ties in the ones before it. `arrange(year, month, day, dep_time)` puts the earliest year first; within a year, the earliest month (January before December); within a month, the earliest day; within a day, the earliest departure time (05:00 before 22:00). The result is chronological order.

```
flights |> arrange(year, month, day, dep_time) |> nrow() # same table, chronological
```

```
#> [1] 336776
```

```
flights |> arrange(desc(dep_time))
```

```
#> # A tibble: 336,776 x 19
#>   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
#>   <int> <int> <int>   <int>         <int>       <dbl>   <int>         <int>
#> 1  2013    10    30    2400           2359         1     327           337
#> 2  2013    11    27    2400           2359         1     515           445
#> 3  2013    12     5    2400           2359         1     427           440
#> # i 336,773 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
#> #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
#> #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

`desc()` wraps exactly one column. `desc(dep_time)` puts the latest departure times first.

The printout shows only the columns that fit, so after sorting by `distance` you may not see `distance` or `dest` at all. Pipe into `glimpse()` to see every column of the top rows, or open the result with `View()`.

```
flights |> arrange(desc(distance)) |> glimpse()
```

```
#> Rows: 336,776
#> Columns: 19
#> $ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
#> $ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
#> $ day       <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
#> $ dep_time  <int> 857, 909, 914, 900, 858, 1019, 1042, 901, 641, 859, 855~
#> $ sched_dep_time <int> 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, ~
#> $ dep_delay <dbl> -3, 9, 14, 0, -2, 79, 102, 1, 1301, -1, -5, 1, -4, -1, ~
#> $ arr_time  <int> 1516, 1525, 1504, 1516, 1519, 1558, 1620, 1504, 1242, 1~
#> $ sched_arr_time <int> 1530, 1530, 1530, 1530, 1530, 1530, 1530, 1530, 1530, 1~
#> $ arr_delay <dbl> -14, -5, -26, -14, -11, 28, 50, -26, 1272, -41, -48, -3~
#> $ carrier   <chr> "HA", "HA", "HA", "HA", "HA", "HA", "HA", "HA", "HA", "HA", "~
#> $ flight    <int> 51, 51, 51, 51, 51, 51, 51, 51, 51, 51, 51, 51, 51, 51, ~
#> $ tailnum   <chr> "N380HA", "N380HA", "N380HA", "N384HA", "N381HA", "N385~
#> $ origin    <chr> "JFK", "JFK", "JFK", "JFK", "JFK", "JFK", "JFK", "JFK", "~
#> $ dest      <chr> "HNL", "HNL", "HNL", "HNL", "HNL", "HNL", "HNL", "HNL", ~
#> $ air_time  <dbl> 659, 638, 616, 639, 635, 611, 612, 645, 640, 633, 613, ~
#> $ distance  <dbl> 4983, 4983, 4983, 4983, 4983, 4983, 4983, 4983, 4983, 4~
#> $ hour      <dbl> 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9~
#> $ minute    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
#> $ time_hour <dtm> 2013-01-01 09:00:00, 2013-01-02 09:00:00, 2013-01-03 0~
```

7 Rows: `distinct()` and `count()`

`distinct()` lists each unique combination of the named columns once. `count()` adds how many rows each has, in a column `n`; `sort = TRUE` puts the largest first.

```
flights |> distinct(origin, dest)
```

```
#> # A tibble: 224 x 2
#>   origin dest
#>   <chr> <chr>
#> 1 EWR    IAH
#> 2 LGA    IAH
#> 3 JFK    MIA
#> # i 221 more rows
```

```
flights |> distinct(year, month, day) |> nrow()
```

```
#> [1] 365
```

```
flights |> count(origin, dest, sort = TRUE)
```

```
#> # A tibble: 224 x 3
#>   origin dest      n
#>   <chr> <chr> <int>
#> 1 JFK    LAX   11262
#> 2 LGA    ATL   10263
#> 3 LGA    ORD    8857
#> # i 221 more rows
```

```
flights |> count(dest, sort = TRUE)
```

```
#> # A tibble: 105 x 2
#>   dest      n
#>   <chr> <int>
#> 1 ORD   17283
#> 2 ATL   17215
#> 3 LAX   16174
#> # i 102 more rows
```

New York flew 224 routes in 2013, on every one of the 365 days. JFK to Los Angeles was the busiest route and Chicago O'Hare the busiest destination. `count()` is the first question to ask of any text column: which values occur, and are any of them typos?

8 Check yourself

From the exercises of Chapter 3. The answers are at the end of the practice exercise.

1. Flights with an arrival delay of two or more hours.
2. Flights that flew to Houston, "IAH" or "HOU".
3. Flights operated by United, American or Delta: "UA", "AA", "DL".
4. Flights that departed in summer: July, August, September.
5. Flights that arrived more than two hours late but did not leave late.
6. Sort flights to find the longest departure delays; then the flights that left earliest in the morning.
7. Was there a flight on every day of 2013?
8. Which flights travelled the farthest? Which the least?

9 What you can do now

- see a table: `print()`, `View()`, `head()`, `glimpse()`, `$`
- write a condition with `==` `!=` `>` `>=` `<` `<=` `&` `|` `! %in%` `is.na()`
- chain steps with the pipe `|>` and read them as “then”
- rows: `filter()`, `arrange()` with `desc()`, `distinct()`, `count()`
- keep a result with `<-`, and know that a verb never changes its input

The next lectures continue with dplyr: columns, then groups.

10 Sources

- Wickham, H., Çetinkaya-Rundel, M. and Grolemund, G. (2023). *R for Data Science*, 2nd ed. Ch. 3, Data transformation, sections Introduction to Rows. Free at <https://r4ds.hadley.nz/data-transform>.
- Wright, C., Ellis, S., Hicks, S. and Peng, R. (2021). *Tidyverse Skills for Data Science in R*, Ch. 3. <https://jhudatascience.org/tidyversecourse/>
- Course repository: <https://github.com/touseefhameed/oka2015-applied-data-science>